

Rendu individuel

Elyess Rjafellah
CI/CD & automatisation

Auteur	Elyess Rjafellah
Rôle	CI/CD & automatisation
Période	Janvier – Juin 2026
Projet	Plateforme de gestion d'un cabinet d'audioprothèse

SUP DE VINCI — Expert en Systèmes d'Information — Promotion 2025-2026. Document réalisé dans le cadre du projet d'étude du Mastère DevOps. Code promo : **CODEPROMO** (à compléter par l'équipe).

1. Ma contribution au projet

Ma responsabilité a porté sur la chaîne d'intégration et de déploiement continu. L'ambition était que l'intégralité du cycle de vie — provisionnement, construction, analyse, déploiement et sauvegarde — s'exécute automatiquement, sans intervention manuelle, et se rétablisse seul en cas d'aléa.

J'ai conçu les différents workflows GitHub Actions : celui d'intégration qui valide le code par le linting et les tests, celui de sécurité qui exécute les analyses, celui de déploiement complet qui orchestre l'ensemble de la mise en production, et enfin un workflow de sauvegarde planifié quotidiennement. J'ai aussi automatisé la création de l'emplacement de stockage de l'état Terraform, de sorte que rien ne doive être préparé à la main au préalable.

Une part importante de mon travail a consisté à fiabiliser ce pipeline face aux situations intermédiaires : un déploiement interrompu pouvait laisser un verrou sur l'état de l'infrastructure ou une livraison Helm dans un état instable. J'ai introduit des mécanismes d'auto-réparation qui détectent et corrigent ces situations, ainsi qu'un ordonnancement correct des dépendances afin que les composants nécessaires soient toujours installés avant ceux qui en dépendent.

2. Perspectives d'évolution

La première évolution que je viserais est l'adoption du GitOps, avec un outil comme ArgoCD, afin que l'état du cluster soit piloté de manière déclarative par le contenu du dépôt plutôt que par des commandes de déploiement. J'introduirais également une authentification fédérée OIDC pour supprimer tout secret de longue durée.

Je mettrais ensuite en place des environnements distincts — développement, pré-production et production — avec une promotion contrôlée des versions, ainsi que des tests de bout en bout exécutés automatiquement après chaque déploiement pour valider la non-régression.

3. Analyse critique des limites techniques rencontrées

Le fait de regrouper le provisionnement et le déploiement dans un workflow unique est pratique pour la démonstration mais allonge la durée d'exécution ; un découpage en étapes parallélisées serait plus efficace. Par ailleurs, l'authentification par identifiant et mot de passe impose un compte sans double authentification, ce qui n'est pas adapté à un contexte de production.

Enfin, l'absence d'environnement de pré-production fait que les essais se déroulent directement sur l'unique cluster, et un déclenchement sur chaque envoi de code pourrait lancer des déploiements coûteux : un garde-fou mériterait d'être ajouté.

4. Annexes

Documentation utilisateur (mon périmètre).

Pour l'utilisateur, tout passe par l'onglet « Actions » du dépôt : le workflow de déploiement met l'application en production, le même workflow permet de tout détruire en choisissant l'action de

destruction, et le workflow de sauvegarde gère la sauvegarde comme la restauration. Les secrets requis sont décrits dans la documentation d'installation.

Analyse personnelle.

Mon principal défi a été de rendre le pipeline robuste : à mesure que je rencontrais des échecs liés à des états intermédiaires, j'ai progressivement ajouté les correctifs nécessaires jusqu'à obtenir une chaîne qui se rétablit d'elle-même.

J'ai gagné en aisance sur GitHub Actions, l'automatisation de Terraform et le diagnostic des pipelines. Je retiens comme axe de progression la modularisation des workflows et la mise en place d'une véritable stratégie multi-environnements associée au GitOps.